TDD in the Agentic Era

What we keep, what we leave behind

Sebastian Larsson

knowit

WHO AM I

Sebastian Larsson

/ Developer

/ Trainer

/ Manager

Certifying developers in TDD since 2014

> **“I’m not a great programmer; I’m just a good programmer with great habits.”**
>
> KENT BECK

knowit

TDD is a habit. Agents don’t have habits, they have instructions.

I no longer think agents should do TDD the way I taught it.

TODAY

Agenda

1 **The old days**

What TDD is, why it works for humans, and a live demo

2 **The new ways**

What happened when agents were told to do TDD

3 **The way forward**

What to leave behind, and what to do instead

PART 1

The old days

What TDD is, and why it works for humans

knowit

THE OLD DAYS

TDD myths

~~"TDD is about testing"~~

~~"TDD is writing the tests first"~~



"You don't write 'the tests' first. You write a test first."

TIM OTTINGER

"TDD is not really about writing tests before code. It's about using tests in short-lived RGR cycles as a tool to design code."

ANDREA LAFORGIA

"The tests that drive our design often correspond to good regression tests, but certainly not always."

JASON GORMAN

knowit

THE OLD DAYS

Definition

→ A development technique

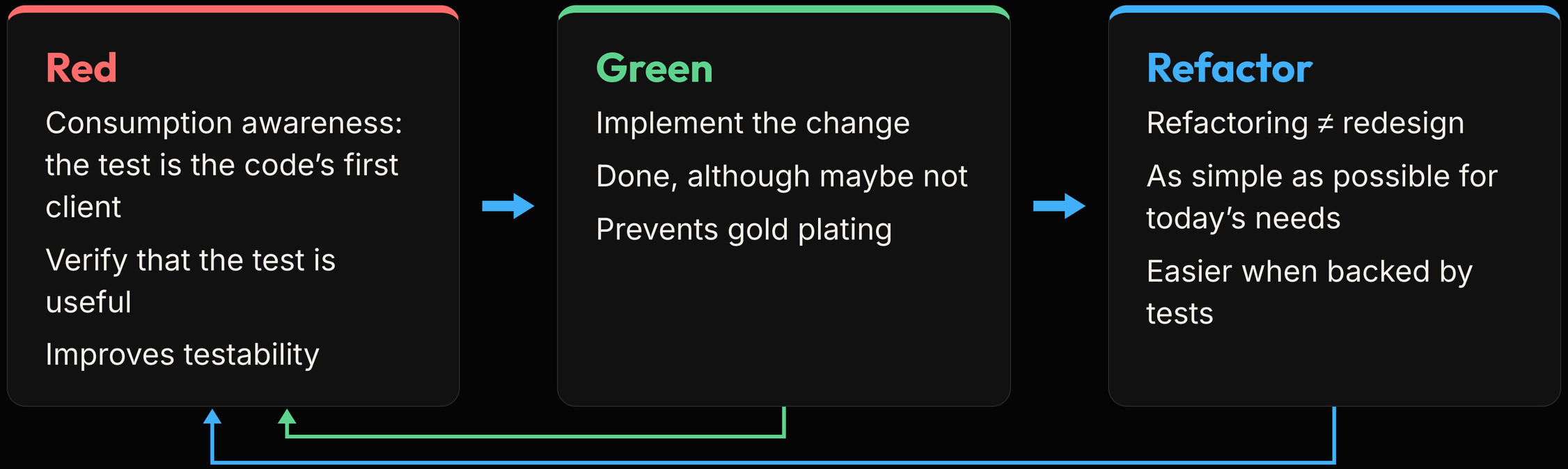
→ Breaking a problem into pieces

→ Rapid feedback

→ A design process

THE OLD DAYS

Why TDD works for humans



A cycle built around human limitations

THE OLD DAYS

Red: create a failing test



Consumption awareness

Verify that the test is useful

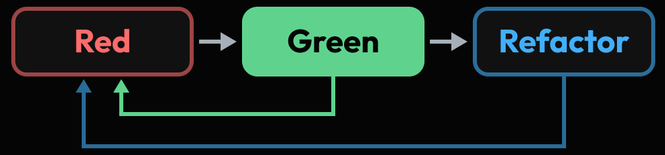
Improves testability

```
@Test
public void simpleAddition() {
    Calculator calculator = new Calculator();
    int expected = 3;
    int actual = calculator.add(1, 2);
    assertThat(actual, is(expected));
}
```

```
@Test
public void detects_adjacent_mine() {
    Mineplacer mineplacer = new Mineplacer() {
        public String constructMineField() {
            return "[*,*,*,*]"
               + "[*,X,*,*]"
               + "[*,*,*,*]"
               + "[*,*,*,*]";
        }};
    MineField mineField = new MineField(mineplacer);
    assertEquals(1, mineField.atPos(0, 0));
}
```

THE OLD DAYS

Green: make the test pass



- → Implement the change
- → Means done, although maybe not
- → Prevents gold plating



THE OLD DAYS

Gold plating

```java
interface MessageStrategy {
    public void sendMessage();
}
abstract class AbstractStrategyFactory {
    public abstract MessageStrategy createStrategy(MessageBody mb);
}
class MessageBody {
    Object payload;
    public Object getPayload() {
        return payload;
    }
    public void configure(Object obj) {
        payload = obj;
    }
    public void send(MessageStrategy ms) {
        ms.sendMessage();
    }
}
public class MyProgram {
    public static void main(String[] args) {
        MessageBody mb = new MessageBody();
        mb.configure("Hello World!");
        AbstractStrategyFactory asf = DefaultFactory.getInstance();
        MessageStrategy strategy = asf.createStrategy(mb);
        mb.send(strategy);
    }
}
class DefaultFactory extends AbstractStrategyFactory {
    private DefaultFactory() { }
    static DefaultFactory instance;

    public static AbstractStrategyFactory getInstance() {
        if (instance == null)
            instance = new DefaultFactory();
        return instance;
    }
    public MessageStrategy createStrategy(final MessageBody mb) {
        return new MessageStrategy() {
            MessageBody body = mb;
            public void sendMessage() {
                Object obj = body.getPayload();
                System.out.println((String) obj);
            }
        };
    }
}
```

46

lines of Java to print “Hello World!”, with a strategy, an abstract factory and a singleton

THE OLD DAYS

Refactor as needed

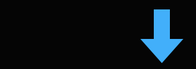


→ Refactoring ≠ redesign

→ As simple an implementation as possible for today's needs

→ Easier when backed by tests

```
// ...
Person p = getPerson(socialSecNr);
//print the person
System.out.println(p.getFirstName());
System.out.println(p.getLastName());
System.out.println(p.getSocialSecNr());
```



```
// ...
Person person = getPerson(socialSecNr);
print(person);
```

BRAIN DUMPING GROUND

Get it out of your head and into the code.

→ Less cognitive load (context)

→ Executable examples

→ Sharper focus

MERCILESS REFACTORING

~~FEAR~~

The tests have your back.

- → Freedom to refactor and experiment
- → Safely make changes

knowit

THE OLD DAYS

The todo list

→ A dynamic buffer that keeps noise away

→ Examples of test cases

→ **Test order has a huge impact on TDD flow**

TODO

- [ ] Equal cards: Tie
- [ ] Steve wins 1 to 0
- [ ] Josh wins x to y
- [ ] J beats T
- [ ] A beats K

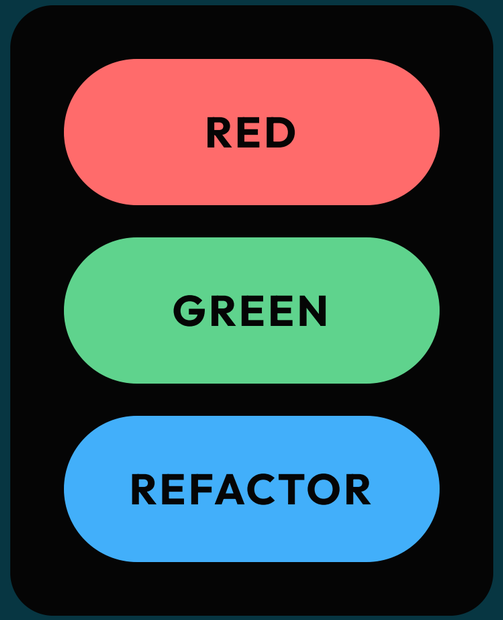
LIVE DEMO

The cards game

codewars.com/kata/53417de006654f4171000587

Steve vs Josh: the higher card wins the round.

“Steve wins x to y”, “Josh wins x to y” or “Tie”



knowit

THE OLD DAYS

The evidence for humans

40–90%

fewer defects in four industrial teams (Nagappan et al. 2008, Microsoft and IBM)

15–35%

more development time, as the teams themselves estimated it

→ Fucci et al. 2017: short, steady cycles mattered more than writing the test first

→ Ghafari et al. 2020: studies disagree mostly because they’re set up differently

Every step answers a human limitation. Which of them does an agent actually have?

PART 2

The new ways

What happened when agents were told to do TDD

knowit

THE NEW WAYS · AN AGENT ≠ A HUMAN

Which of TDD's benefits does an agent need?

It doesn't have

Fear

Boredom

Context switching fatigue

Not knowing where the code is going

It has, as badly as us or worse

Misreading the spec

Claiming done when it isn't

Blind spots in its own tests

Gold plating

Discipline inconsistency

In my experiment, one agent wrote the same misreading into its test and its code.

THE NEW WAYS

What others found

BIRGITTA BÖCKELER · AUG 2026

TDD prompts gave no quality gain, for 3–8.5× the tokens.

EMILY BACHE · SEP 2026

That tested a prompt, not TDD. A test list and a harness are what's missing.

→ **So I tested both: TDD by prompt, and TDD enforced by a harness.**

THE NEW WAYS

The experiment

Setup	Runs
No TDD	Opus 5.5
TDD by prompt	Opus 5 (twice), DeepSeek 4.1 Flash, GLM 5.3 Flash, one unrecorded agent
TDD by prompt, local	Qwen, Gemma 4
TDD by prompt, plan agreed first	Astra
TDD enforced by a harness	Opus 5.5 with probity
TDD by hand, live	Me, 2016, in front of an audience

→ Every red and green replayed and analyzed from the git history

9 + 1

agent runs on the same kata: 9 told to do TDD, 1 told not to

9

planted bugs via mutation testing, plus an acceptance suite the agents never saw

THE NEW WAYS

Agentic TDD skills: it looks good, but it’s a charade

46%

of tests passed as soon as they were written (26 of 57): the agent already knew the answer

→ GLM: 7 of 7 genuine reds, every commit labelled correctly, and it ships “Josh wins 1 to 2”

→ Ten implementations, one design: a rank string, indexOf, two counters

→ On this kata, following TDD didn’t change the agents’ design. It only tidied it.

THE NEW WAYS

Tiny steps are less meaningful for an agent

Fake it	Increases cycle speed for humans. An agent doesn’t need this detour.
Minimal steps	Lower a human’s risk. An agent doesn’t need to play it this safe.
Gorman called it	The tests that drive the design aren’t always good regression tests
“A test first, not the tests”	A human rule against overinvesting up front: GLM wrote all 7 tests first, as skipped tests. For an agent, that investment is cheap.

THE NEW WAYS

Enforcing TDD with a harness: probity

11 of 12

tests genuinely red

4

real refactors: the most faithful TDD in the repo

→ The same result as no TDD, at 13× the cost and 22× the time

→ It dropped the tests that couldn’t go red, so it caught fewer planted bugs: 6 of 9, against 7 of 9 without TDD

→ 9 of 13 blocks were false positives; resending the same edit usually got through

THE NEW WAYS

The cost: tokens, dollars and time

Run	Time	Cost	Planted bugs caught
No TDD · Opus 5.5	29 s	$0.27	7 of 9
TDD by prompt · Opus 5, run 1	3 min 47 s	$2.07	8 of 9
TDD by prompt · Opus 5, run 2	4 min 18 s	$2.72	5 of 9
TDD enforced by probity · Opus 5.5	10 min 48 s	$3.54	6 of 9

Small experiment: one run per setup, on an easy 20-line kata

8–13× the cost and 8–22× the time, with no measurable quality gain.

Are harnesses worth the cost?

Model	Harness	Accepted	Time	Cost
Sol (frontier)	No	10 of 10	6 min 51 s	~$0.41
Sol (frontier)	Yes	10 of 10	19 min 53 s	~$2.68
Luna (cheap)	No	3 of 10	15 min 56 s	~$0.09
Luna (cheap)	Yes	10 of 10	35 min	~$0.27

→ His harness isn’t TDD: human-written specs and examples, plus an independent acceptance gate

Frontier model: one independent check at the end. Cheap model: checks throughout.

PART 3

The way forward

For the coming year

knowit

THE WAY FORWARD · FOR THE AGENT

What we keep, what we leave behind

Keep

- ✓ Examples agreed before any code
- ✓ A check the agent didn’t write
- ✓ A failing test first for every bug
- ✓ Refactoring on green

Leave behind

- ~~Fake it and minimal green~~
- ~~Forcing the smallest step~~
- ~~A perfect process as proof of quality~~
- ~~Red as the proof that a test can fail~~

→ **Keep the proof, drop the ritual: mutation-test afterwards instead (Gorman, Böckeler)**

THE WAY FORWARD

Planning

Why? To prevent rework, and to make sure interpretations are correct.

THIS TALK WAS PLANNED FIRST

~76%

of the planned slides' text is still the first generated version

The ambiguity

GLM's "Josh wins 1 to 2": no agent's TDD caught the "x to y" ambiguity.

One agreed example, "Josh wins 2 to 1", would have caught it in seconds.

The expectations

Agreed with the team before the agent starts.

Astra's were agreed with me first, and it's the only TDD suite that pins the "x to y" reading.

The agent turns them into its own test list.

A small step compared to spec-driven development (spec-kit, OpenSpec)

THE WAY FORWARD

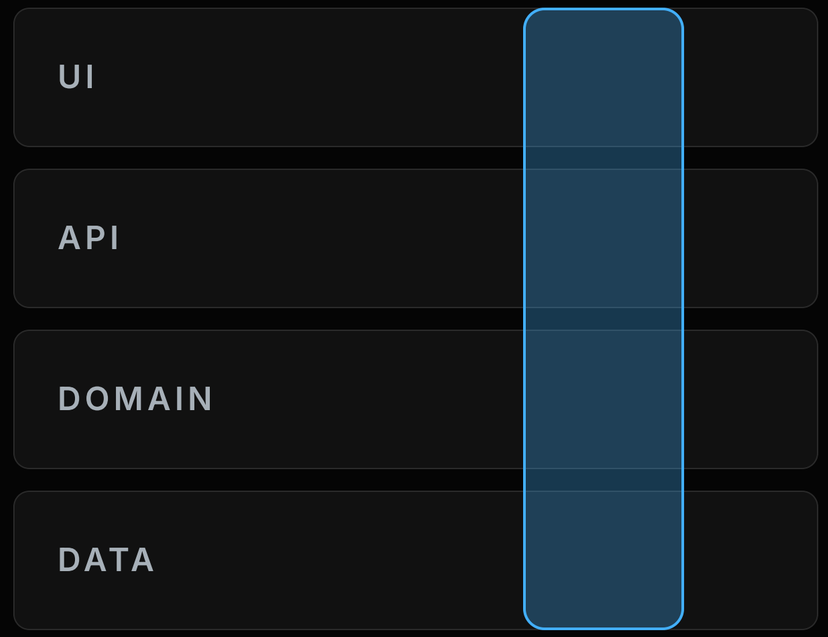
Vertical slicing

→ Small diffs a human can actually review

→ Fast incremental delivery at all layers

From experience, not from this experiment

> **“On anything non-trivial, the spec is never right. … Adapting to feedback is the real requirements discipline.”**
>
> JASON GORMAN



ONE SLICE THROUGH EVERY LAYER

THE WAY FORWARD

Move up a loop



The double loop from Growing Object-Oriented Software (Freeman & Pryce, 2009)

→ Outer loop, acceptance tests: the human

→ Inner loop, agent decides

> **“A human still owns what ships but now they’re reviewing evidence and not just diffs.”**
> ADDY OSMANI

Just ATDD/BDD again? Yes. But the examples stay with you.

THE WAY FORWARD

Use acceptance-test-driven examples to specify desired behaviour

THIS EXPERIMENT

Only tests the same agent didn't write caught GLM's and Gemma's bugs

MATHEWS & NAGAPPAN 2024

Tests given with the task help LLMs

KONSTANTINOU ET AL. 2026

14% vs 25%

faults found by tests written after faulty code vs. independent tests

→ Jain's harness is the same idea, tested at scale

Examples are executable; a spec document isn't.

THE WAY FORWARD · ANDREA LAFORGIA, MAR 2026

Expectation-Driven Development

1. Write the expectations in plain text, before any code
2. The agent proves each one with executed evidence, not a story about it
3. Critical paths become automated tests

> **“The fox can guard the henhouse if there’s a different fox checking the first fox’s work.”**
>
> ANDREA LAFORGIA

His own caveat: not battle-tested yet

Test-first becomes example-first.

From Beck's TDD by Example to Adzic's Specification by Example.

Write the expectations as BDD-style examples before any code.
Let the agent decide everything below them.
Have something the agent didn't write check the result.

Questions?

Sebastian Larsson

knowit

LINKS

References

Beck (2002) *Test-Driven Development: By Example* · book
Nagappan et al. (2008) Realizing quality improvement through test driven development · microsoft.com/en-us/research (PDF)
Fucci et al. (2017) A dissection of the test-driven development process · arxiv.org/abs/1611.05994
Ghafari et al. (2020) Why research on test-driven development is inconclusive · arxiv.org/abs/2007.09863
Böckeler (2026) TDD inside the agent loop · martinfowler.com/articles/exploring-gen-ai/tdd-in-the-agent-loop.html
Bache (2026) Response video, Modern Software Engineering · youtube.com/watch?v=wK5Wgbqtl50
Selander probity: TDD hooks for coding agents · github.com/nizos/probity
Jain (2026) Are harnesses worth the cost? · martinfowler.com/articles/are-harnesses-worth-the-cost.html
Freeman & Pryce (2009) *Growing Object-Oriented Software, Guided by Tests* · book
Gorman (2026) The spec is never right (LinkedIn) · lnkd.in/p/d3ZyQ4hs
Osmani (2026) Agentic verification · addyo.substack.com/p/agentic-verification
Mathews & Nagappan (2024) Test-driven development for code generation · arxiv.org/abs/2402.13521
Konstantinou, Tambon & Papadakis (2026) On the risk of coding before testing · arxiv.org/abs/2607.05139
Laforgia (2026) Expectation-Driven Development · a4al6a.substack.com/p/expectation-driven-development-a
Adzic (2011) *Specification by Example* · book
This experiment all runs, branches and analysis · github.com/00Zeb/simple-card-game